

Energy Efficiency in Railway Scheduling

Paul Tennigkeit, Georgi Abshilava

Universität Potsdam

21.09.2026

Outline

Motivation: Energy Efficiency Problem

Baseline ASP Model

Acceleration Mechanics

Energy Mechanics

Results and Comparison with Baseline Model

Conclusion

Motivation: Energy Efficiency Problem

Baseline ASP Model

- ▶ Establishes rules to handle essential train behavior
 - ▶ Directions
 - ▶ Movements
 - ▶ Transitions
 - ▶ Collision and Reachability Constraints
 - ▶ Schedule Optimization

Directions and Movements

% Directions

dir(s).

dir(n).

dir(w).

dir(e).

% Movement Operation

moving(move_forward).

moving(move_left).

moving(move_right).

% Movement on Grid

delta(A,s,1,0) :- moving(A).

delta(A,n,-1,0) :- moving(A).

delta(A,e,0,1) :- moving(A).

delta(A,w,0,-1) :- moving(A).

Transitions

```
% Transition facts: To represent different track types
% ID : track ID
% Dir1: pointing direction of agent
% Act : associated action with movement towards exit
% Dir2: exit direction
transition(ID, Dir1, Act, Dir2).
```

Collision Constraints

```
% Trains can not occupy the same cell at the same time
:- stay(ID1,X,Y,T),
   stay(ID2,X,Y,T),
   ID1 < ID2.
```

```
% Trains can not swap their position
:- stay(ID1,X1,Y1,T),
   stay(ID2,X2,Y2,T),
   stay(ID1,X2,Y2,T+1),
   stay(ID2,X1,Y1,T+1),
   ID1 < ID2.
```

Reachability Constraints

```
% define states (cell, time, direction),  
% which are reachable by a train  
reachable(ID,T,A,X2,Y2,T2,Dir2) :-  
    A != wait,  
    position(ID,(X,Y),T,Dir1),  
    cell((X,Y),Cell),  
    transition(Cell,Dir1,A,Dir2),  
    delta(A,Dir2,DX,DY),  
    X2 = X + DX,  
    Y2 = Y + DY,  
    speed(ID,S),  
    T2 = T + S,  
    time(T2).  
  
reachable(ID,T,wait,X,Y,T+1,Dir) :-  
    position(ID,(X,Y),T,Dir),  
    time(T+1).
```

Reachability Constraints

```
% Choose exactly one valid transition at each time step
1 { position(ID,(X2,Y2),T2,Dir2)
    : reachable(ID,T,A,X2,Y2,T2,Dir2)
} 1 :- position(ID,(_,_),T,_),
    not reached(ID,T).

% Track the time interval a train spends inside a cell.
% This is to make enforcing collision constraints easier.
chosen(ID,T,A,X,Y,X2,Y2,T2) :-
    position(ID,(X,Y),T,_),
    position(ID,(X2,Y2),T2,_),
    reachable(ID,T,A,X2,Y2,T2,_).

% a train reached its target if their position matches
reached(ID,T) :- position(ID,(X,Y),T,_),
    end(ID,(X,Y),_).
```

Schedule Optimization

```
% stop searching if train is not
% in time to reach its target anymore
:- position(ID,(X,Y),T,_),
   end(ID,(EX,EY),Arr),
   speed(ID,InvSpd),
    $T + (|X-EX| + |Y-EY|)*InvSpd > Arr.$ 

% the arrival times should be minimized
reached_earlier(ID,T) :- reached(ID,T),
                           reached(ID,T0),  $T0 < T.$ 
arrival(ID,T) :- reached(ID,T),
                  not reached_earlier(ID,T).

% minimize summed arrival times
#minimize { T@2,ID : arrival(ID,T) }.
```

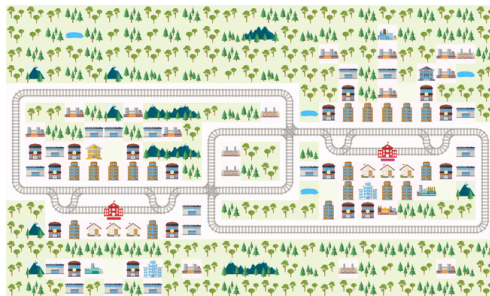


Figure: Environment used to test the encodings

Table: Train 0

Time step	Position	direction	status	Given Command
24	(8, 10)	s	moving	move_forward
25	(9, 10)	s	moving	move_right
26	(9, 9)	w	moving	move_forward
27	(9, 8)	w	moving	move_left
28	(10, 8)	s	moving	move_forward
29	(10, 7)	w	moving	move_forward
30	(10, 6)	w	moving	move_forward
31	(10, 5)	w	moving	none

Table: Train 1

Time step	Position	direction	status	Given Command
12	(6, 13)	e	moving	move_forward
13	(6, 14)	e	moving	move_forward
14	(6, 15)	e	moving	move_forward
15	(6, 16)	e	moving	move_right
16	(7, 16)	s	moving	move_forward
17	(7, 17)	e	moving	move_forward
18	(7, 18)	e	moving	move_forward
19	(7, 19)	e	moving	none

Acceleration Mechanics

- ▶ Speed Changes in General
- ▶ Turning, Waiting, and Arrival Physics
- ▶ Solver Heuristics

Speed Changes in General

```
% Interpret given train speeds as maximum speeds  
max_speed(ID,S) :- speed(ID,S).
```

```
% Trains start with a speed of 1/4  
temp_speed(ID, Dep+1, 4) :- start(ID,(_,_),Dep,_).
```

Speed Changes in General

```
% Allow gradual speed changed, while
% respecting the physical limits of the train.
% The train can change its speed up to one
% step at a time
1{
    temp_speed(ID,T2,S-1);
    temp_speed(ID,T2,S);
    temp_speed(ID,T2,S+1)
}1
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 >= MS,
   S+1 <= 4.
```

Speed Changes in General

```
% Lower Bound
1{
    temp_speed(ID,T2,S-1);
    temp_speed(ID,T2,S)
}1
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 >= MS,
   S+1 > 4.
```

Speed Changes in General

```
% Upper Bound
1{
    temp_speed(ID,T2,S);
    temp_speed(ID,T2,S+1)
}1
:- chosen(ID,T,A,_,_,_,_,T2),
   A != wait,
   temp_speed(ID,T,S),
   max_speed(ID,MS),
   S-1 < MS,
   S+1 <= 4.
```

Speed Changes in General

```
% In between (maintain speed)
temp_speed(ID,T2,S) :- chosen(ID,T,A,_,_,_,_,T2),
                        A != wait,
                        temp_speed(ID,T,S),
                        max_speed(ID,MS),
                        S-1 < MS,
                        S+1 > 4.
```

Turning, Waiting, and Arrival Physics

```
% You have to take a turn with inverse speed  $\geq 3$ 
:- position(ID,_,T,Dir1),
   temp_speed(ID,T,S),
   T2 = T + S,
   position(ID,_,T2,Dir2),
   Dir1  $\neq$  Dir2,
   S < 3.
```

Turning, Waiting, and Arrival Physics

```
% to be able to wait, you need to have speed 1/4
reachable(ID,T,wait,X,Y,T+1,Dir) :-
    position(ID,(X,Y),T,Dir),
    time(T+1),
    temp_speed(ID,T,4).

% after waiting, you have to start with speed 1/4
temp_speed(ID,T2,4) :- chosen(ID,T,wait,_,_,_,T2).
```

Turning, Waiting, and Arrival Physics

```
% you can only arrive at your goal if your  
% inverse speed was 4 before  
:- reached(ID,T2),  
    temp_speed(ID,T1,S),  
    T2 = T1 + S,  
    S < 4.
```

Solver Heuristics

```
% prefer to not wait
wait_position(ID,X2,Y2,T2,Dir2) :-
    reachable(ID,T,wait,X2,Y2,T2,Dir2).

#heuristic position(ID,(X2,Y2),T2,Dir2)
    : wait_position(ID,X2,Y2,T2,Dir2).
    [-50,1]

% schedule trains one by one
#heuristic position(ID,(X,Y),T,Dir) : start(ID,_,Dep,_).
    [-Dep,1]

% prefer getting faster
#heuristic temp_speed(ID,T,S) : position(ID,_,T,_).
    [-S,1]
```

Energy Mechanics

- ▶ Energy Cost Factors and Power Types
- ▶ Energy Consumption for Different Types of Actions (Braking, Turning, Coasting)
- ▶ Encoding Optimization

Cost Factors and Power Types

```
% Energy cost factor of trains
% based on power type + weight
pow_cost_fac(ID,1) :- pow_type(ID, electro).
pow_cost_fac(ID,2) :- pow_type(ID, hybrid).
pow_cost_fac(ID,3) :- pow_type(ID, diesel).
pow_cost_fac(ID,4) :- pow_type(ID, hydrogen).

% A train's weight is defined in
% the environment's logic program
cost_fac(ID,CF) :- pow_cost_fac(ID,PCF),
                    weight(ID,W),
                    CF = PCF * W.
```

Cost of Braking

```
% Tracks whether the train was waiting
% in the previous time step
prev_wait(ID,T) :- action(train(ID),_,T),
                    time(T-1),
                    action(train(ID),wait,T-1).

energy(ID,T+1,E) :- action(train(ID),wait,T),
                    speed(ID,S),
                    not prev_wait(ID,T),
                    cost_fac(ID,CF),
                    E = CF * 24 / S.
```

Cost of Coasting

```
energy(ID,T2,E) :- action(train(ID),A,T),  
                    A != wait,  
                    position(ID,_,T,Dir),  
                    speed(ID,S),  
                    T2 = T + S,  
                    position(ID,_,T2,Dir),  
                    cost_fac(ID,CF),  
                    E = CF.
```

Cost of Braking

```
energy(ID,T2,E) :- position(ID,_,T,Dir1),  
                    speed(ID,S),  
                    T2 = T + S,  
                    position(ID,_,T2,Dir2),  
                    Dir1 != Dir2,  
                    cost_fac(ID,CF),  
                    E = CF * 12 / S.
```

Encoding Optimization

```
% Optimize for energy efficiency  
% Arrival time still has higher priority than energy  
#minimize { E@1,ID,T : energy(ID,T,E) }.
```

Results and Comparison with Baseline Model

Conclusion